

Technology Stack

Date	03 July 2026
Team ID	
Project Name	Credit Card Approval Prediction
Maximum Marks	2 Marks

Define Your Technology Stack:

Identify the programming languages, frameworks, databases, front-end tools, back-end tools, and APIs that you will use to build your solution. A well-chosen technology stack ensures that your application is scalable, maintainable, and aligned with your project requirements.

Fill out the table below to detail the specific technologies chosen for each component of your architecture and provide a brief justification for your choice.

Technology Stack Details

S.No	Architecture Component / Layer	Technology Chosen	Justification / Purpose
1	Frontend / Client-Side (e.g., React, Angular, HTML/CSS)	HTML5, CSS3, Jinja2 Templates (via Flask)	HTML5 and CSS3 are used to build the web application interface — the single applicant prediction form and batch screening page. Jinja2 (Flask's built-in templating engine) dynamically renders prediction results on the same page without needing a separate frontend framework, keeping the stack simple and lightweight.
2	Backend / Server-Side (e.g., Node.js, Python/Django, Java)	Python 3.14 Flask 3.x	Python is the primary language for the entire project — data processing, model training, and web server. Flask is used as the backend web framework to handle HTTP routes (/predict for single prediction, /batch and /batch_predict for CSV batch screening). Flask was chosen for its simplicity, lightweight nature, and seamless integration with Python ML libraries.
3	Database / Data Storage (e.g., MySQL, MongoDB, PostgreSQL)	CSV Files (Kaggle Dataset) Joblib PKL Files	The project uses CSV files as the primary data source — application_record.csv (438,557 applicant records) and credit_record.csv (1M+ monthly payment records) from Kaggle. Processed train/test splits are saved as CSV files. The trained XGBoost model and feature columns are stored as .pkl files using Joblib for fast loading in the Flask application. No relational database is needed since predictions are stateless.

S.No	Architecture Component / Layer	Technology Chosen	Justification / Purpose
4	Cloud / Hosting / Deployment (e.g., AWS, Azure, Heroku, Docker)	IBM Watson Machine Learning (IBM Cloud)	IBM Watson Machine Learning is used for cloud deployment of the trained XGBoost model. The deployment pipeline (documented in 05_watson_deployment.ipynb) uploads the model to IBM Watson ML repository, creates an online scoring endpoint, and enables real-time predictions accessible via REST API from anywhere — making the solution scalable beyond the local Flask server.
5	Version Control & CI/CD (e.g., GitHub, GitLab, Jenkins)	GitHub Visual Studio Code (VS Code)	GitHub is used for version control — storing all project files including Jupyter notebooks, app.py, HTML templates, and model files. The project is developed in Visual Studio Code with the Jupyter extension, providing an integrated environment for running notebooks, editing Python files, and managing the terminal. VS Code's Explorer panel gives a clear view of the entire project structure.
6	Third-Party APIs / Other Tools (e.g., Stripe, Twilio, Google Maps)	Scikit-learn XGBoost Pandas NumPy Matplotlib Seaborn Joblib	• Scikit-learn — Logistic Regression, Decision Tree, Random Forest training and evaluation • XGBoost — Best-performing gradient boosting classifier (78.9% accuracy, 62% recall) • Pandas — Data loading, merging, cleaning, and feature engineering • NumPy — Numerical operations and array handling • Matplotlib & Seaborn — EDA visualizations: income distribution, correlation heatmap, class imbalance chart, model comparison bar chart • Joblib — Saving and loading trained model (.pkl) and feature columns